

Lab 5: Potential Outcomes, Confounding, Colliders, and DAGs

SOCIOL 723, Social Statistics II, Thursday, September 24

Wenhao Jiang

Table of contents

1	Goals	2
2	Potential outcomes on a population you built	2
2.1	Both columns, then the switching equation	2
2.2	Randomization	3
2.3	Selection on observables: the adjustment formula by hand	4
3	Confounding	5
4	Colliders	6
4.1	Berkson's paradox, directly	7
5	Post-treatment control	8
6	The mechanism test: Baron and Kenny (1986)	9
6.1	The same test in a package, plus the sensitivity analysis	10
7	Bias amplification	12
8	Selection bias	13
9	DAGs in R	13
9.1	Testable implications	15
10	Drawing DAGs	15
11	Exercises	17
12	Session information	18

1 Goals

The point of this lab is that **you already know the right answer**. We build worlds where every unit has both potential outcomes, so the ATE is a number we chose, then see which analyses recover it and which do not.

1. Build a population with both potential outcomes; compute the ATE, ATT, ATU; verify the selection-bias decomposition.
2. Randomize, then apply the adjustment formula by hand, and watch both work.
3. Simulate confounding, colliders, mediators, bias amplification, and selection; run Baron and Kenny's mechanism test on the GSS and break it.
4. Draw and query DAGs: `dagitty` for the logic, TikZ for the picture.

```
library(dplyr)
library(ggplot2)
library(dagitty)
library(ggdag)
library(mediation) # mediate, medsens
set.seed(723)
```

2 Potential outcomes on a population you built

2.1 Both columns, then the switching equation

Two thousand people. $X_i = 1$ if the parents went to college. The untreated outcome is higher for $X_i = 1$, and the treatment effect is *lower* for $X_i = 1$: the negative-selection pattern of Brand and Xie (2010) from lecture, built in.

```
n <- 2000
X <- rbinom(n, 1, 0.5)
Y0 <- 10 + 4 * X + rnorm(n, 0, 2)      ## baseline: 4 higher if X = 1
tau <- 6 - 4 * X + rnorm(n, 0, 1)      ## effect: 6 if X = 0, 2 if X = 1
Y1 <- Y0 + tau
pop <- data.frame(X, Y0, Y1, tau)
head(pop, 4)
#>   X   Y0   Y1   tau
#> 1 0 12.37 19.24 6.8753
#> 2 1 12.42 15.87 3.4475
#> 3 1 11.88 15.46 3.5770
#> 4 1 17.19 17.73 0.5433

## the estimands are averages of a column nobody ever sees in full
ATE <- mean(pop$tau)
ATE
#> [1] 4.026
```

Now assign treatment with selection: $X_i = 1$ is treated with probability 0.7, $X_i = 0$ with probability 0.3. The switching equation reveals one column per person.

```
pop$D <- rbinom(n, 1, ifelse(pop$X == 1, 0.7, 0.3))
pop$Y <- pop$D * pop$Y1 + (1 - pop$D) * pop$Y0      ## the switching equation

## ATT and ATU exist because WE still see both columns; a researcher does not
ATT <- mean(pop$tau[pop$D == 1]); ATU <- mean(pop$tau[pop$D == 0])
pi <- mean(pop$D)
c(ATE = ATE, ATT = ATT, ATU = ATU, check = pi * ATT + (1 - pi) * ATU)
#>      ATE      ATT      ATU check
#> 4.026 3.278 4.738 4.026
```

The naive comparison and its decomposition from lecture, term by term:

```
naive <- mean(pop$Y[pop$D == 1]) - mean(pop$Y[pop$D == 0])
baseline <- mean(pop$Y0[pop$D == 1]) - mean(pop$Y0[pop$D == 0])  ## E[Y0|D=1] - E[Y0|D=0]
c(naive = naive, ATT_plus_baseline = ATT + baseline,
  ATE_plus_both = ATE + baseline + (1 - pi) * (ATT - ATU))
#>      naive ATT_plus_baseline      ATE_plus_both
#> 4.696      4.696      4.696
```

Both identities hold to the last decimal. The treated have a higher baseline (they are mostly $X_i = 1$) and a smaller effect, and the naive comparison adds the first error to the ATT.

```
## regression of Y on D returns the naive comparison exactly; next week opens
## with why (the error term holds the baseline and the excess gain)
coef(lm(Y ~ D, data = pop))["D"]
#>      D
#> 4.696
```

2.2 Randomization

Same population, treatment by coin flip. Now D is independent of both columns, the baseline term is zero on average, and the difference in means is centred on the ATE.

```
pop$Dr <- rbinom(n, 1, 0.5)
pop$Yr <- pop$Dr * pop$Y1 + (1 - pop$Dr) * pop$Y0
c(estimate = mean(pop$Yr[pop$Dr == 1]) - mean(pop$Yr[pop$Dr == 0]), ATE = ATE,
  baseline_gap = mean(pop$Y0[pop$Dr == 1]) - mean(pop$Y0[pop$Dr == 0]))
#>      estimate      ATE baseline_gap
#> 4.08127      4.02611      0.01555
```

The estimate misses the ATE by sampling noise only; the baseline gap is close to zero because a coin does not read $Y_i(0)$. The figure at the end of this section repeats the draw 1,000 times.

2.3 Selection on observables: the adjustment formula by hand

Back to the selected treatment D. Within each value of X, treatment was a coin flip (probability 0.7 or 0.3, unrelated to the potential outcomes), so conditional independence holds by construction. The lecture's formula: difference within each stratum, then weight by the share of the stratum.

```
cell <- pop |>
  group_by(X) |>
  summarise(share = n() / nrow(pop),
            diff = mean(Y[D == 1]) - mean(Y[D == 0]), ## CATE(x), estimable
            true_cate = mean(tau), .groups = "drop")
cell
#> # A tibble: 2 x 4
#>       X share diff true_cate
#>   <int> <dbl> <dbl>     <dbl>
#> 1     0 0.509  6.03       5.97
#> 2     1 0.491  1.97       2.01
c(adjusted = sum(cell$diff * cell$share), ATE = ATE)
#> adjusted      ATE
#>    4.036    4.026

## the regression version. It agrees with the formula up to sampling noise
## because Var(D | X) is the same in both strata (0.7 * 0.3 = 0.3 * 0.7), so
## OLS weights the two CATEs by the stratum shares. Week 6 shows what happens
## when the variances differ (exercise 3)
coef(lm(Y ~ D + X, data = pop))["D"]
#>      D
#> 4.003
```

Repeat the whole thing 1,000 times with fresh treatment draws, for all three designs.

```
three <- t(replicate(1000, {
  D <- rbinom(n, 1, ifelse(pop$X == 1, 0.7, 0.3)); Y <- D * pop$Y1 + (1 - D) * pop$Y0
  Dr <- rbinom(n, 1, 0.5); Yr <- Dr * pop$Y1 + (1 - Dr) * pop$Y0
  cell <- sapply(0:1, function(x) mean(Y[D == 1 & pop$X == x]) - mean(Y[D == 0 & pop$X == x]))
  c(`selection, naive` = mean(Y[D == 1]) - mean(Y[D == 0]),
    `randomized, naive` = mean(Yr[Dr == 1]) - mean(Yr[Dr == 0]),
    `selection, adjusted` = sum(cell * table(pop$X) / n))
}))
as.data.frame(three) |>
  tidyr::pivot_longer(everything(), names_to = "design") |>
  mutate(design = factor(design, levels = colnames(three))) |>
  ggplot(aes(value)) +
  geom_histogram(bins = 40, fill = "grey75") +
  geom_vline(xintercept = ATE, linetype = 2) +
```

```

facet_wrap(~ design) +
labs(x = "estimate of the ATE", y = NULL) +
theme_minimal(base_size = 10)
round(rbind(mean = colMeans(three), sd = apply(three, 2, sd)), 3)
#>      selection, naive randomized, naive selection, adjusted
#> mean          4.765          4.028          4.030
#> sd            0.096          0.103          0.104

```

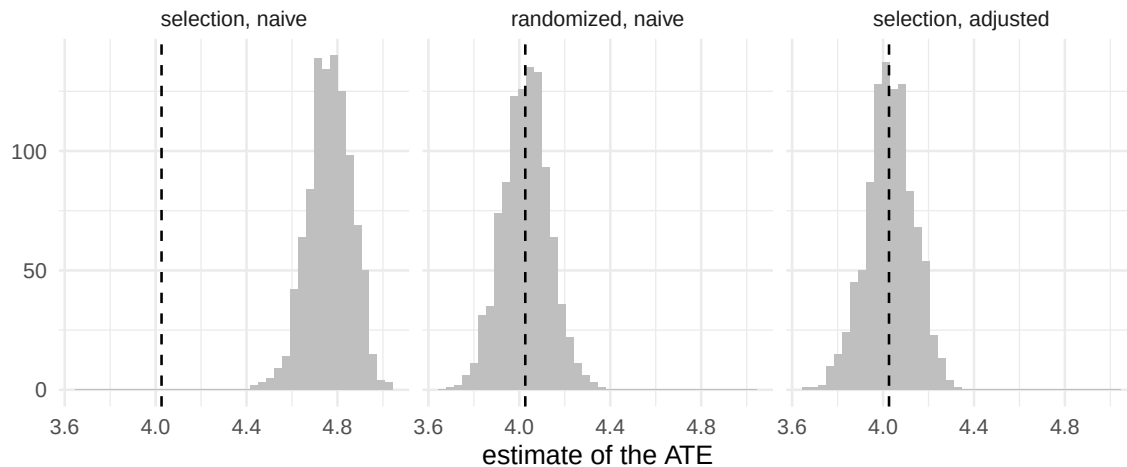


Figure 1: One population, treatment redrawn 1,000 times. The naive comparison under selection is centred on the wrong number; randomization and the adjustment formula are centred on the ATE (dashed). All three have about the same spread: bias is not imprecision.

3 Confounding

The world: X causes both D and Y ; the true effect of D on Y is $\tau = 2$. This is the fork of the lecture's three building blocks, and the back-door path $D \leftarrow X \rightarrow Y$ is the only one.

```

sim_confound <- function(n = 2000, tau = 2) {
  x <- rnorm(n)                ## confounder
  d <- 1.2 * x + rnorm(n)       ## treatment
  y <- tau * d + 1.5 * x + rnorm(n) ## outcome
  data.frame(y, d, x)
}
dat <- sim_confound()

c(naive      = coef(lm(y ~ d, data = dat))["d"],
  adjusted   = coef(lm(y ~ d + x, data = dat))["d"],
  truth      = 2)
#>      naive.d adjusted.d      truth
#>      2.732      1.984      2.000

```

The naive estimate is biased upward. Check the omitted variable bias formula from Week 1:

```
b_x <- coef(lm(y ~ d + x, data = dat))["x"]  ## effect of the omitted variable
delta <- coef(lm(x ~ d, data = dat))["d"]    ## omitted on included
c(predicted_bias = b_x * delta,
  actual_bias = coef(lm(y ~ d, data = dat))["d"] - coef(lm(y ~ d + x, data = dat))["d"])
#> predicted_bias.x    actual_bias.d
#>          0.7482          0.7482
```

4 Colliders

Now the world has **no confounding at all**: D and Y share no common cause. But both cause C .

```
sim_collider <- function(n = 2000, tau = 2) {
  d <- rnorm(n)
  y <- tau * d + rnorm(n)
  c_ <- 1.0 * d + 1.0 * y + rnorm(n)  ## collider: caused by BOTH
  data.frame(y, d, c = c_)
}
dat2 <- sim_collider()

c(correct = coef(lm(y ~ d, data = dat2))["d"],
  controlled = coef(lm(y ~ d + c, data = dat2))["d"],
  truth = 2)
#>   correct.d controlled.d      truth
#>    2.0289    0.5794    2.0000
```

! Important

Read that table again. The **unadjusted** regression is right; adding a control makes it wrong. Nothing about the data would tell you this. Only the DAG does.

```
reps2 <- t(replicate(1000, {
  d0 <- sim_collider()
  c(correct = coef(lm(y ~ d, d0))["d"], controlled = coef(lm(y ~ d + c, d0))["d"])
}))

as.data.frame(reps2) |>
  tidyr::pivot_longer(everything()) |>
  ggplot(aes(value, fill = name)) +
  geom_histogram(bins = 60, alpha = .7, position = "identity") +
  geom_vline(xintercept = 2, colour = "black", linetype = 2) +
  scale_fill_manual(values = c("correct.d" = "navy", "controlled.d" = "firebrick")) +
```

```
labs(x = expression(hat(tau)), y = "count", fill = NULL) +
theme_minimal(base_size = 10)
```

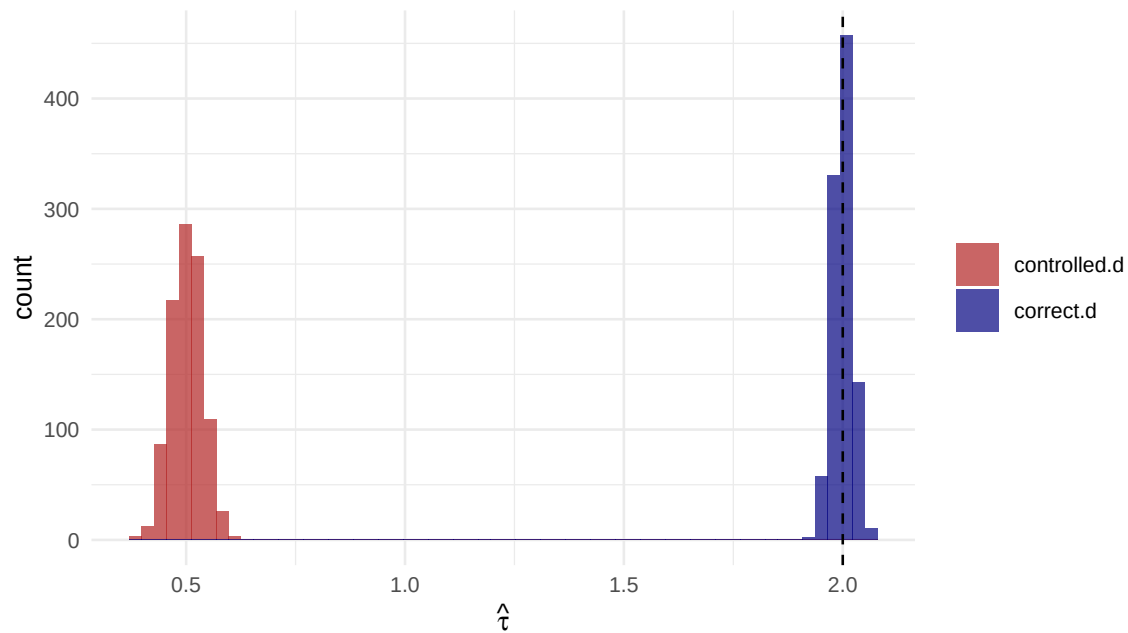


Figure 2: Controlling for a collider introduces bias where none existed.

4.1 Berkson's paradox, directly

Two genuinely independent traits become correlated once you condition on their common effect. This is the admissions example from lecture.

```
n_b <- 5000
test <- rnorm(n_b); athl <- rnorm(n_b)
admit <- (test + athl) > 1.5      ## admitted if either is high enough
```

```
c(all_applicants = cor(test, athl),
  admitted_only = cor(test[admit], athl[admit]))
#> all_applicants admitted_only
#>      -0.006449      -0.668406
```

```
tibble::tibble(test, athl, admit) |>
  ggplot(aes(test, athl, colour = admit)) +
  geom_point(alpha = .25, size = .5) +
  geom_smooth(method = "lm", se = FALSE, linewidth = .7) +
  scale_colour_manual(values = c("grey70", "navy")) +
  labs(colour = "admitted") +
  theme_minimal(base_size = 10)
```

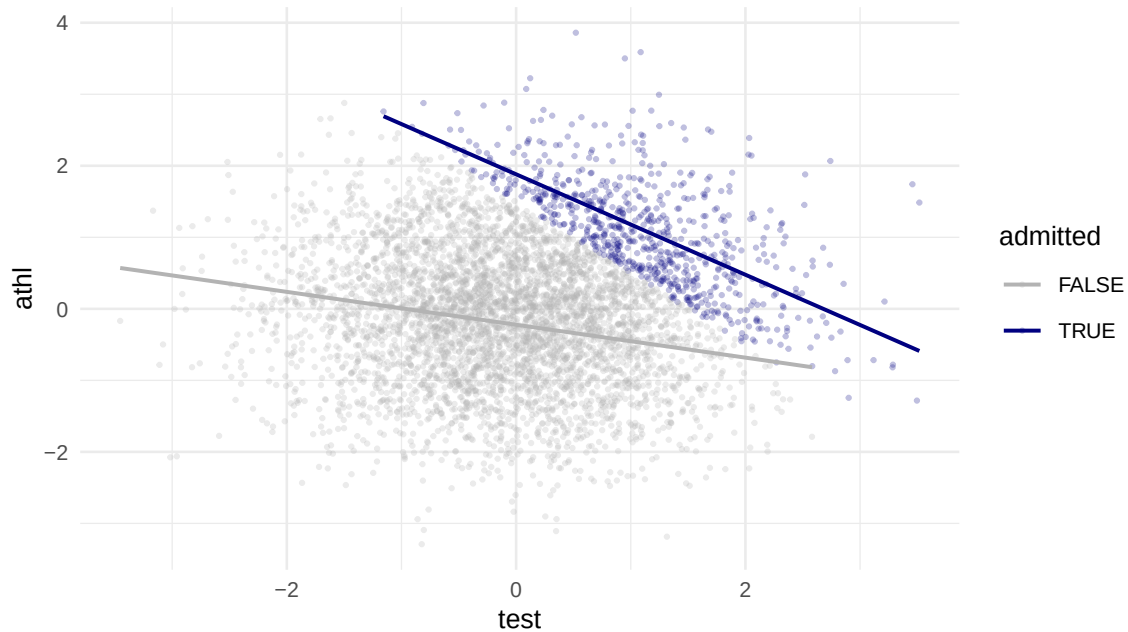


Figure 3: Test score and athletic talent are independent among applicants and negatively related among admitted students.

5 Post-treatment control

D affects Y both directly and through a mediator M . Controlling for M answers a different question.

```
sim_med <- function(n = 2000) {
  d <- rnorm(n)
  m <- 0.8 * d + rnorm(n)          ## mediator
  y <- 1.0 * d + 1.5 * m + rnorm(n) ## total effect = 1.0 + 0.8*1.5 = 2.2
  data.frame(y, d, m)
}
dat3 <- sim_med()

c(total_effect = coef(lm(y ~ d, data = dat3))["d"],
  direct_effect = coef(lm(y ~ d + m, data = dat3))["d"],
  truth_total = 2.2,
  truth_direct = 1.0)
#> total_effect.d direct_effect.d truth_total truth_direct
#>          2.259          1.044          2.200          1.000
```

Both numbers are “correct”, they estimate *different estimands*. The mistake is not computing the second; it is reporting it as though it were the first.

Warning

This is the single most common error in applied sociology: controlling for occupation when studying the effect of education on earnings, controlling for income when studying the effect of education on health, controlling for marriage when studying the effect of a policy on fertility.

6 The mechanism test: Baron and Kenny (1986)

The lecture's three regressions, on the mediator world above. The truth is $a = 0.8$, $b = 1.5$, $c' = 1$, so the indirect effect is $ab = 1.2$ and the total effect is $c = 2.2$.

```
bk <- function(dat) {
  m1 <- lm(y ~ d, data = dat)          ## step 1: total effect c
  m2 <- lm(m ~ d, data = dat)          ## step 2: D moves M, coefficient a
  m3 <- lm(y ~ d + m, data = dat)      ## step 3: b for M, c' for D
  a <- coef(m2)["d"]; b <- coef(m3)["m"]
  ## Sobel SE for ab: delta method, a and b from different regressions so
  ## their covariance is taken as zero
  se_ab <- sqrt(b^2 * vcov(m2)["d", "d"] + a^2 * vcov(m3)["m", "m"])
  c(c = coef(m1)["d"], a = a, b = b, c_prime = coef(m3)["d"],
    ab = a * b, se_ab = se_ab, share = a * b / coef(m1)["d"])
}
round(bk(dat3), 3)
#>      c.d      a.d      b.m c_prime.d      ab.d      se_ab.m      share.d
#>      2.259      0.793      1.531      1.044      1.215      0.039      0.538
## the identity c = c' + ab holds exactly for OLS: it is the omitted-variable
## formula with M as the omitted variable
with(as.list(bk(dat3)), c(c = c.d, c_prime_plus_ab = c_prime.d + ab.d))
#>      c c_prime_plus_ab
#>      2.259      2.259
```

Now the same test on real data: parents' education, own education, log earnings, with sex, race, and age as covariates in every equation.

```
gss <- readRDS("../Data/gss_earnings.rds")
c_tot <- coef(lm(lnearn ~ pareduc + female + black + age, data = gss))["pareduc"]
a <- coef(lm(educ ~ pareduc + female + black + age, data = gss))["pareduc"]
f <- coef(lm(lnearn ~ pareduc + educ + female + black + age, data = gss))
round(c(c = c_tot, a = a, b = f["educ"], c_prime = f["pareduc"],
  ab = a * f["educ"], share = a * f["educ"] / c_tot), 4)
#>      c.pareduc      a.pareduc      b.educ c_prime.pareduc      ab.pareduc
#>      0.0625      0.3546      0.1108      0.0232      0.0393
#>      share.pareduc
```

```
#> 0.6290
```

A paper would write: “63 percent of the effect of parents’ education on earnings runs through the respondent’s own schooling.” Step 3 is a regression of earnings on schooling, and the lecture’s back-door question applies to it: is there an unmeasured cause of both own education and earnings? Ability is one. Nothing in the three regressions can detect it. Build it in and watch:

```
sim_med_u <- function(n = 20000) {
  d <- rnorm(n); u <- rnorm(n)          ## u: unmeasured, into M and Y
  m <- 0.8 * d + u + rnorm(n)
  y <- 1.0 * d + 1.5 * m + u + rnorm(n)  ## same a, b, c' as before
  data.frame(y, d, m)
}
round(rbind(truth = c(c = 2.2, a = 0.8, b = 1.5, c_prime = 1, ab = 1.2, se_ab = NA, share = 1),
            baron_kenny = bk(sim_med_u()), 3)
#>           c      a      b c_prime   ab se_ab share
#> truth      2.200 0.80 1.500   1.000 1.200    NA 0.545
#> baron_kenny 2.183 0.79 2.005   0.598 1.584 0.021 0.726
```

The total effect c and the first-stage a survive, because D has no back door. The mediator coefficient b absorbs U , so the indirect effect is overstated, the direct effect understated, and the “share mediated” moves from 55 to 73 percent. The identity $c = c' + ab$ still holds; only the split is wrong, and the split was the point. Modern mediation analysis names the assumption sequential ignorability (Imai, Keele, and Tingley 2010) and pairs the estimate with a sensitivity analysis for U .

6.1 The same test in a package, plus the sensitivity analysis

The `mediation` package takes the mediator model and the outcome model and returns the indirect effect (it calls it the ACME, average causal mediation effect), the direct effect (ADE), and the share mediated, with simulation intervals. With linear models and no interaction the ACME is ab and the ADE is c' , so this is Baron and Kenny with better intervals. What the package adds is `medsens()`: how strong would the unmeasured U have to be to erase the indirect effect?

```
m_med <- lm(educ ~ pareduc + female + black + age, data = gss)      ## step 2
m_out <- lm(l_earn ~ pareduc + educ + female + black + age, data = gss)  ## step 3
## sims: draws of the coefficients for the intervals (Week 3's parametric bootstrap)
med <- mediate(m_med, m_out, treat = "pareduc", mediator = "educ", sims = 500)
summary(med)
#>
#> Causal Mediation Analysis
#>
#> Quasi-Bayesian Confidence Intervals
#>
```

```

#>           Estimate 95% CI Lower 95% CI Upper           p-value
#> ACME           0.0391      0.0347      0.0439 <0.0000000000000002 ***
#> ADE            0.0232      0.0150      0.0307 <0.0000000000000002 ***
#> Total Effect    0.0623      0.0545      0.0704 <0.0000000000000002 ***
#> Prop. Mediated  0.6258      0.5464      0.7330 <0.0000000000000002 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Sample Size Used: 3509
#>
#>
#> Simulations: 500

## sensitivity: rho is the correlation between the errors of the mediator and
## outcome equations that an unmeasured U would induce. At what rho is ACME 0?
sens <- medsens(med, rho.by = 0.1, sims = 200)
summary(sens)
#>
#> Mediation Sensitivity Analysis for Average Causal Mediation Effect
#>
#> Sensitivity Region
#>
#>      Rho    ACME 95% CI Lower 95% CI Upper R^2_M*R^2_Y* R^2_M~R^2_Y~
#> [1,] 0.3 0.0029      -0.001      0.0067      0.09      0.0548
#>
#> Rho at which ACME = 0: 0.3
#> R^2_M*R^2_Y* at which ACME = 0: 0.09
#> R^2_M~R^2_Y~ at which ACME = 0: 0.0548

```

The ACME of 0.039 and ADE of 0.023 are the ab and c' computed by hand. The sensitivity table says the indirect effect reaches zero when the error correlation is 0.3, or when an unmeasured U explains about 9 percent of the variance in both schooling and earnings. Whether ability is that strong is the substantive question; the package puts a number on it where Baron and Kenny said nothing.

```
plot(sens, main = "", ylim = c(-0.02, 0.06))
```

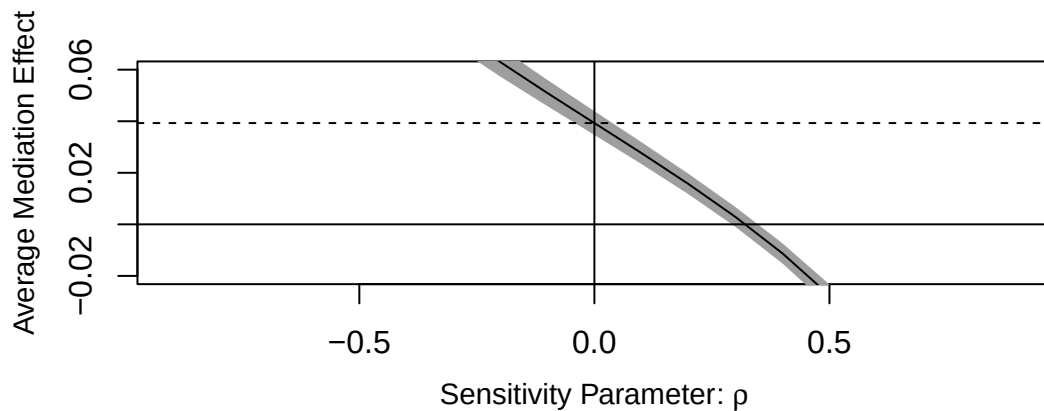


Figure 4: Sensitivity of the indirect effect to an unmeasured confounder of schooling and earnings.

7 Bias amplification

The lecture's third single-cause case: Z causes D only, and an unmeasured U confounds D and Y . Z is not on any back-door path, so controlling for it cannot remove bias. It makes the bias worse.

```
sim_amp <- function(n = 5000, tau = 1) {
  z <- rnorm(n); u <- rnorm(n)          ## z: cause of D only; u: unmeasured confounder
  d <- 1.5 * z + u + rnorm(n)
  y <- tau * d + 2 * u + rnorm(n)
  data.frame(y, d, z)
}
dat5 <- sim_amp()
c(without_z = coef(lm(y ~ d, data = dat5))["d"],
  with_z    = coef(lm(y ~ d + z, data = dat5))["d"],
  truth     = 1)
#> without_z.d    with_z.d      truth
#>      1.485      2.015      1.000
```

Why: by FWL (Week 1), controlling for Z regresses Y on the part of D that Z does not explain. That residual is $U + \epsilon_D$, in which U is a *larger* share than it was in D . The clean variation from Z was the part of D that was safe to use, and the control threw it away. Variables like Z are instruments (Week 7).

8 Selection bias

Now the sample itself is selected on a collider. Only people who are employed appear in the data, and employment depends on both the treatment and the unobserved determinants of wages.

```
sim_select <- function(n = 5000, tau = 1) {  
  educ <- rnorm(n)  
  u <- rnorm(n) ## unobserved ability  
  wage <- tau * educ + u + rnorm(n)  
  employed <- (0.8 * educ + 1.2 * u + rnorm(n)) > 0.5  
  data.frame(wage, educ, employed)  
}  
d4 <- sim_select()  
  
c(full_population = coef(lm(wage ~ educ, data = d4))["educ"],  
  employed_only = coef(lm(wage ~ educ, data = d4, subset = employed))["educ"],  
  truth = 1)  
#> full_population.educ    employed_only.educ          truth  
#>                1.0138                0.7363                1.0000
```

The regression on the employed is biased **downward**, even though the specification is the one that was unbiased in the full population. Restricting to the employed conditions on a collider, so unobserved ability becomes correlated with education *within the sample*. The sample restriction *is* the conditioning.

! Important

`gss_earnings.rds` is built this way: `build_gss_extract.R` keeps respondents working full time with positive earnings. Every earnings regression in this course runs on a sample selected on a descendant of schooling and of the unobserved determinants of earnings. Say so when you report one.

9 DAGs in R

`dagitty` lets you state a graph and ask it what to condition on. This is the lecture's worked example: degree D , earnings Y , family background X , a measured test score A , unmeasured school quality U , and occupation M as one channel from degree to earnings.

```
g <- dagitty('dag {  
  U [unobserved]  
  X -> D    X -> Y  
  A -> D    A -> Y  
  U -> A    U -> Y  
  D -> M    M -> Y
```

```

D -> Y
D [exposure] Y [outcome]
}')

adjustmentSets(g, exposure = "D", outcome = "Y", effect = "total")
#> { A, X }

ggdag(tidy_dagitty(g, layout = "sugiyama"), text_size = 3, node_size = 12) +
  theme_dag_blank()

```

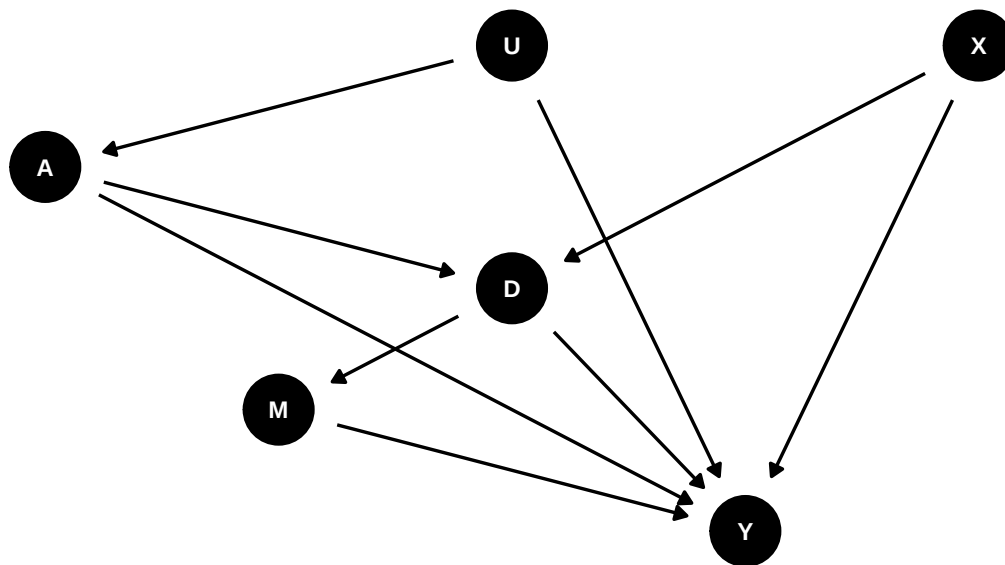


Figure 5: The lecture's college-and-earnings DAG, drawn by ggdag.

Ask it directly whether a candidate adjustment set works, and why not:

```

isAdjustmentSet(g, c("X", "A"), exposure = "D", outcome = "Y")
#> [1] TRUE
isAdjustmentSet(g, "X", exposure = "D", outcome = "Y")           ## A's back door open
#> [1] FALSE
isAdjustmentSet(g, c("X", "A", "M"), exposure = "D", outcome = "Y") ## mediator
#> [1] FALSE

```

Every path, and which of them $\{X, A\}$ leaves open:

```

paths(g, from = "D", to = "Y")$paths
#> [1] "D -> M -> Y"      "D -> Y"          "D <- A -> Y"      "D <- A <- U -> Y"
#> [5] "D <- X -> Y"
paths(g, from = "D", to = "Y", Z = c("X", "A"))$open    ## TRUE = still open
#> [1] TRUE  TRUE FALSE FALSE FALSE

```

Only the two causal paths, $D \rightarrow Y$ and $D \rightarrow M \rightarrow Y$, stay open under $\{X, A\}$: the back-door criterion, checked by machine. U needed no adjustment because every path through it passes through A, a chain node that A blocks.

9.1 Testable implications

A DAG implies conditional independencies among the *observed* variables. Those are testable, and they are the only part of a DAG that data can speak to.

```
impliedConditionalIndependencies(g)
#> A _||_ M | D
#> A _||_ X
#> M _||_ X | D

## simulate from the DAG and check
sim <- simulateSEM(g, N = 5000)
localTests(g, sim, type = "cis.loess", R = 50) |> round(3)
#>           estimate std.error  2.5% 97.5%
#> A _||_ M | D   -0.018    0.015 -0.046 0.010
#> A _||_ X       0.005    0.014 -0.025 0.027
#> M _||_ X | D   -0.015    0.013 -0.040 0.006
```

i Note

A DAG that passes its testable implications is not thereby “correct”: many different DAGs imply the same independencies. A DAG that *fails* them is wrong, and that is worth knowing. Note what is absent from the list: nothing about $Y(d)$. Identification assumptions are not among the testable implications.

10 Drawing DAGs

ggdag is fine for a quick look. For a paper or a slide you want control over placement, and the standard tool is TikZ. Every DAG in this week’s slides is drawn with one style block and a handful of commands, and the same code works in a Quarto PDF document. The style, placed once in the preamble (here, in the YAML `include-in-header`), defines the node shape, the arrow, and three variants:

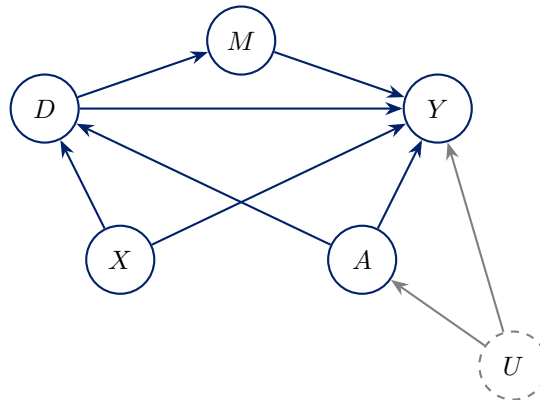
```
\usepackage{tikz}
\usetikzlibrary{arrows.meta,positioning}
\tikzset{
  dag/.style={>=Stealth,thick,
    every node/.style={circle,draw=dukeblue,fill=white,inner sep=1pt,
      minimum size=9mm,font=\small}},
  unobs/.style={draw=gray,dashed}, % unmeasured variable
  cond/.style={draw=accent,fill=accent!12}, % a variable conditioned on
```

```
lab/.style={draw=none,fill=none,rectangle,inner sep=2pt,font=\footnotesize},
arr/.style={->,dukeblue,thick},
uarr/.style={->,gray,thick}} % arrow from an unmeasured variable
```

A picture is then a list of nodes at coordinates, and a list of arrows between them. This is the college-and-earnings DAG, exactly as it appears in the slides:

```
\begin{tikzpicture}[dag]
  \node (D) at (0,0)    {$D$};
  \node (M) at (2.6,0.9) {$M$};
  \node (Y) at (5.2,0)  {$Y$};
  \node (X) at (1.0,-2.0) {$X$};
  \node (A) at (4.2,-2.0) {$A$};
  \node[unobs] (U) at (6.2,-3.4) {$U$};
  \draw[arr] (D)--(M); \draw[arr] (M)--(Y); \draw[arr] (D)--(Y);
  \draw[arr] (X)--(D); \draw[arr] (X)--(Y);
  \draw[arr] (A)--(D); \draw[arr] (A)--(Y);
  \draw[uarr] (U)--(A); \draw[uarr] (U)--(Y);
\end{tikzpicture}
```

In a Quarto document, put that code inside a raw LaTeX block, which starts with ````\l=latex` and is passed straight to the PDF engine. The result:

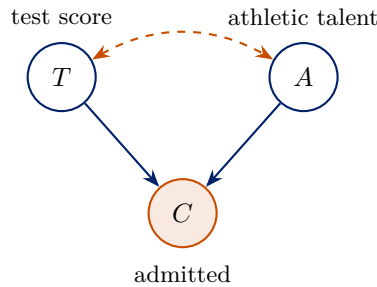


Three more things you will want. A conditioned node uses `\node[cond]`; a curved arrow uses `to[bend left=30]` in place of `--`; a text label uses `\node[lab]`. The collider example from lecture uses all three:

```
\begin{tikzpicture}[dag]
  \node (T) at (0,0)    {$T$}; \node (A) at (3.2,0) {$A$};
  \node[cond] (C) at (1.6,-1.8) {$C$};
  \draw[arr] (T)--(C); \draw[arr] (A)--(C);
  \draw[<->,accent,dashed,thick] (T) to[bend left=30] (A);
  \node[lab] at (0,0.8) {test score}; \node[lab] at (3.2,0.8) {athletic talent};
  \node[lab] at (1.6,-2.6) {admitted};
```



```
\end{tikzpicture}
```



The same code goes into a beamer frame unchanged; add `scale=0.85`, `every node/.append style={transform shape}` to the options to shrink a large graph. The complete style is in the YAML header of this file and in `slides.tex`.

11 Exercises

1. **A precision variable.** Simulate $Z \rightarrow Y$ with no arrow into D , plus $D \rightarrow Y$. Compare the estimate and its standard error with and without Z over 500 replications. Which case of the lecture's single-cause frame is this, and does the DAG say to include Z ?
2. **Positivity.** Modify `sim_confound()` so that everyone with $x > 1$ is treated with probability 0.99. Estimate the ATE with and without those observations. What does regression do in the region with no comparison cases?
3. **Heterogeneous effects.** In Section 2, change the treatment probabilities to 0.9 for $X_i = 1$ and 0.3 for $X_i = 0$, so that $\text{Var}(D_i | X_i)$ differs across strata. Recompute the adjustment formula and `lm(Y ~ D + X)`. They now disagree; say which one is the ATE and what the other one is. Week 6 derives the weights.
4. **Your own DAG.** Draw the DAG for a causal question in your own research, first in `dagitty` and then in TikZ. Mark unobserved variables. Ask for `adjustmentSets()`. Is there any set you can actually measure? If not, that is a finding, and it points you toward Weeks 7 to 11.
5. **The birth-weight paradox.** Simulate: smoking $\rightarrow$ birth weight, an unobserved severe condition $\rightarrow$ birth weight and $\rightarrow$ infant mortality, smoking $\rightarrow$ mortality (a genuine harmful effect). Show that among low-birth-weight infants, smoking appears protective. Then draw the DAG with birth weight as a conditioned node.
6. **State an estimand.** For the GSS earnings data, write one sentence in the Lundberg, Johnson, and Stewart (2021) form: a unit-specific quantity and a target population. Then say what would have to be true for `lm(lnearn ~ ba + ...)` to estimate it. Remember that the extract keeps only full-time workers (Section 8).

12 Session information

```
sessionInfo()
#> R version 4.5.1 (2025-06-13)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Tahoe 26.6.2
#>
#> Matrix products: default
#> BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
#> other attached packages:
#> [1] mediation_4.5.1 sandwich_3.1-1 mvtnorm_1.3-3 Matrix_1.7-3
#> [5] MASS_7.3-65     ggdag_0.2.13   dagitty_0.3-4  ggplot2_4.0.0
#> [9] dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#> [1] gtable_0.3.6      xfun_0.53         htmlwidgets_1.6.4 ggrepel_0.9.6
#> [5] lattice_0.22-7    Rdpack_2.6.4      vctr_0.6.5        tools_4.5.1
#> [9] generics_0.1.4    curl_7.0.0        tibble_3.3.0      cluster_2.1.8.1
#> [13] pkgconfig_2.0.3   data.table_1.17.8 checkmate_2.3.3    RColorBrewer_1.1-3
#> [17] S7_0.2.0          lifecycle_1.0.4   compiler_4.5.1    farver_2.1.2
#> [21] stringr_1.5.1     ggforce_0.5.0     graphlayouts_1.2.5 htmltools_0.5.8.1
#> [25] yaml_2.3.10       htmlTable_2.4.3   Formula_1.2-5     pillar_1.11.0
#> [29] nloptr_2.2.1      tidyr_1.3.1       cachem_1.1.0      viridis_0.6.5
#> [33] Hmisc_5.2-3       reformulas_0.4.1  rpart_4.1.24      boot_1.3-31
#> [37] nlme_3.1-168      tidyselect_1.2.1  digest_0.6.37     stringi_1.8.7
#> [41] purrr_1.1.0       labeling_0.4.3    splines_4.5.1     polyclip_1.10-7
#> [45] fastmap_1.2.0     grid_4.5.1        colorspace_2.1-2  cli_3.6.5
#> [49] magrittr_2.0.3    base64enc_0.1-3   ggraph_2.2.2      tidygraph_1.3.1
#> [53] foreign_0.8-90    withr_3.0.2       scales_1.4.0      backports_1.5.0
#> [57] rmarkdown_2.29    igraph_2.2.1      nnet_7.3-20       lme4_1.1-37
#> [61] gridExtra_2.3     zoo_1.8-14        memoise_2.0.1     lpSolve_5.6.23
```

```
#> [65] evaluate_1.0.5      knitr_1.50           rbibutils_2.3        V8_8.2.0
#> [69] viridisLite_0.4.2    mgcv_1.9-3           rlang_1.1.6          Rcpp_1.1.0
#> [73] glue_1.8.0           tweenr_2.0.3         rstudioapi_0.17.1    minqa_1.2.8
#> [77] jsonlite_2.0.0       R6_2.6.1
```